



Documentação Técnica:

Sincronização de Sensores

Main-Process & VM

Este documento descreve como o **ZoyBlocks** garante que os dados dos sensores (Cliff/Visão) cheguem ao motor de execução do Blockly em tempo real, atravessando as camadas do Electron.

1. Arquitetura da Ponte de Dados

Para que o robô tome decisões rápidas, os dados seguem este fluxo:

1. **Python (Hardware):** Captura o evento IR (`EvtCliffDetectedChange`).
2. **Main Process (Cérebro):** Intercepta o JSON via `stdout` e atualiza o objeto `global`.
3. **VM (Lógica):** O interpretador lê o objeto `global` instantaneamente ao executar um bloco de "Se".

2. Implementação no Main Process (`main.js`)

Implementamos variáveis de estado no escopo global do Node.js. Isso é vital porque a VM do Blockly não tem acesso ao `window` (DOM), apenas ao ambiente de execução do Node.

JavaScript

```
// Inicialização de Memória de Curto Prazo
global.estadoVisao = { rostoDetectado: false, ultimoQR: "" };
global.estadoSensores = { detectouBorda: false };
```

Sincronização Ativa

O listener do processo Python agora atua como um "despachante" de dados:

- **Filtro JSON:** Faz o parse das strings vindas do Python.
 - **Atualização Global:** Alimenta `global.estadoSensores` e `global.estadoVisao`.
 - **Encaminhamento UI:** Envia os mesmos dados para o terminal visual do usuário.
-

3. Integração com a VM (blockly-service.js)

A Máquina Virtual foi configurada para ser "consciente" do estado do robô. Em vez de enviar um comando para o Python e esperar, ela faz uma **leitura direta de memória**.

Função de Ponte Corrigida:

```
JavaScript
checarBorda: async () => {
  // A VM lê diretamente do estado global sincronizado pelo main.js
  return global.estadoSensores?.detectouBorda || false;
}
```

4. Protocolo de Teste (Validação de Segurança)

Para confirmar que a implementação está correta, siga estes passos:

Passo	Ação	Resultado Esperado	Status
1	Iniciar Cozmo no App	Log "Ready" no terminal e vídeo ativo.	[]
2	Levantar o Robô (Simular Borda)	Terminal Node exibir: [SENSOR] Borda detectada: true.	[]
3	Executar Bloco "Se Borda"	O robô deve reagir imediatamente (Luz/Som/Parada).	[]
4	Colocar Robô no Chão	global.estadoSensores.detectouBorda deve voltar para false.	[]

5. Notas de Manutenção

- **Buffer do Python:** O uso de PYTHONUNBUFFERED: "1" no spawn é obrigatório para evitar atrasos (lag) na detecção de borda.
- **Escopo:** Lembre-se que global no Main Process é diferente de window no Renderer. A VM sempre consulta o global.

Status do Sistema: ☒ Sincronizado e Pronto para Testes de Campo.

Gostaria de prosseguir com a criação de um **Bloco de Interrupção de Emergência** (um código que roda "por trás" e para o robô mesmo que o programa do usuário não tenha um bloco de "Se")?